

# RoboPet — Phase 1 (UI)

Riverpod, App Lifecycle & the Pet Screen (Flutter)

Final Phase 1 step. This wires the finished domain + data layer to the screen: Riverpod providers (DB init -> repository -> reactive watchObject stream), a WidgetsBindingObserver that runs offline catch-up on resume + foreground smoothing, and a clean, functional Pet Screen with animated stat bars and Feed / Oil / Play actions.

## 1. Dependency

Add Riverpod to pubspec.yaml:

```
dependencies:  
  flutter_riverpod: ^2.5.1
```

New files added this step:

```
lib/  
  main.dart          # entry point + ProviderScope override  
  app/  
    providers.dart    # Riverpod graph  
    app_lifecycle_reactor.dart # resume -> reconcile + foreground ticker  
  ui/  
    pet_screen.dart   # the screen + stat bars + actions
```

## 2. Repository: add the three care actions

RobotRepository already has feed(). Generalise it so all three buttons share one time-correct path (settle decay first, THEN add the boost). Replace feed() with this block in lib/data/robot\_repository.dart:

```
/// Which care stat a player action tops up.  
enum CareKind { battery, oil, entertainment }  
  
// --- add inside class RobotRepository ---  
  
/// Settles elapsed decay, THEN adds [amount] to the chosen stat, so the  
/// boost always stacks on the correct (decayed) base value.  
Future<RobotInstance?> applyCare(  
  String instanceId,  
  CareKind kind, {  
    double amount = 60,  
  }) async {  
  final robot = await _isar.robotInstances  
    .filter()  
    .instanceIdEqualTo(instanceId)  
    .findFirst();  
  if (robot == null) return null;
```

```

    final result = _reconcileInPlace(robot); // advance time first

    final stat = switch (kind) {
      CareKind.battery => robot.battery,
      CareKind.oil => robot.oil,
      CareKind.entertainment => robot.entertainment,
    };
    stat.value = (stat.value + amount).clamp(0.0, stat.max).toDouble();
    if (kind == CareKind.battery) robot.lastFedAt = _clock.now().toUtc();

    await _persist(robot, result);
    return robot;
  }

  Future<RobotInstance?> feed(String id, {double amount = 60}) =>
    applyCare(id, CareKind.battery, amount: amount);

  Future<RobotInstance?> clean(String id, {double amount = 60}) =>
    applyCare(id, CareKind.oil, amount: amount);

  Future<RobotInstance?> play(String id, {double amount = 60}) =>
    applyCare(id, CareKind.entertainment, amount: amount);

```

Every write goes through `_persist`, which calls `isar.writeTxn(... put ...)`. Because the UI listens to `watchObject`, the screen rebuilds automatically after each action -- no manual `setState` or stream refresh needed.

### 3. Riverpod providers

#### `lib/app/providers.dart`

```

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:isar/isar.dart';

import '../data/isar_service.dart';
import '../data/models/player_profile.dart';
import '../data/models/robot_instance.dart';
import '../data/robot_factory.dart';
import '../data/robot_repository.dart';
import '../domain/clock.dart';

/// Overridden in main() with the already-open instance (see main.dart).
final isarProvider = Provider<Isar>(
  (ref) => throw UnimplementedError('isarProvider must be overridden in main()'),
);

final clockProvider = Provider<Clock>((ref) => const SystemClock());

final robotRepositoryProvider = Provider<RobotRepository>((ref) {
  return RobotRepository(
    ref.watch(isarProvider),
    clock: ref.watch(clockProvider),
  );
});

/// First-launch seeding + the initial offline catch-up.
/// Returns the active robot's Isar id (the key for watchObject).
final activeRobotIdProvider = FutureProvider<int?>((ref) async {
  final isar = ref.watch(isarProvider);

```

```

final repo = ref.watch(robotRepositoryProvider);
final clock = ref.watch(clockProvider);

// Seed a starter pet + profile the very first time.
if (await isar.robotInstances.count() == 0) {
  final robot = RobotFactory.starter(
    instanceId: 'starter-001',
    now: clock.now(),
  );
  await isar.writeTxn(() async {
    await isar.robotInstances.put(robot);
    await isar.playerProfiles.put(
      PlayerProfile()
        ..playerId = 'local-player'
        ..displayName = 'Player'
        ..activeRobotId = robot.instanceId
        ..ownedRobotIds = [robot.instanceId],
    );
  });
}

// Compute offline progression at launch, return the active id.
final active = await repo.loadActiveAndReconcile();
return active?.id;
});

/// Reactive stream of the active robot. Emits on every DB write
/// (decay tick, care action, offline catch-up) -> UI rebuilds.
final activeRobotStreamProvider = StreamProvider<RobotInstance?>((ref) async* {
  final isar = ref.watch(isarProvider);
  final id = await ref.watch(activeRobotIdProvider.future);
  if (id == null) {
    yield null;
    return;
  }
  yield* isar.robotInstances.watchObject(id, fireImmediately: true);
});

```

## 4. App entry point + lifecycle observer

### lib/main.dart

We open Isar before runApp and inject it via a ProviderScope override, so the rest of the graph is fully synchronous.

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'app/app_lifecycle_reactor.dart';
import 'app/providers.dart';
import 'data/isar_service.dart';
import 'ui/pet_screen.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final isar = await IsarService.instance();

  runApp(
    ProviderScope(
      overrides: [isarProvider.overrideWithValue(isar)],
      child: const RoboPetApp(),
    ),
  );
}

```

```

    );
  }

class RoboPetApp extends StatelessWidget {
  const RoboPetApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'RoboPet',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        brightness: Brightness.dark,
        colorSchemeSeed: Colors.tealAccent,
      ),
      // The reactor wraps the screen so lifecycle events are observed app-wide.
      home: const AppLifecycleReactor(child: PetScreen()),
    );
  }
}

```

### lib/app/app\_lifecycle\_reactor.dart

Runs the offline catch-up on resume, and (for manual real-time verification) advances + persists on a short foreground ticker so the bars visibly move. The ticker is cancelled when the app is backgrounded.

```

import 'dart:async';

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'providers.dart';

class AppLifecycleReactor extends ConsumerStatefulWidget {
  const AppLifecycleReactor({super.key, required this.child});
  final Widget child;

  @override
  ConsumerState<AppLifecycleReactor> createState() =>
    _AppLifecycleReactorState();
}

class _AppLifecycleReactorState extends ConsumerState<AppLifecycleReactor>
  with WidgetsBindingObserver {
  Timer? _ticker;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _startTicker();
  }

  @override
  void dispose() {
    _ticker?.cancel();
    WidgetsBinding.instance.removeObserver(this);
    super.dispose();
  }

  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    switch (state) {
      case AppLifecycleState.resumed:
        _reconcile(); // offline catch-up for the time we were away
    }
  }
}

```

```

        _startTicker(); // resume foreground smoothing
        case AppLifecycleState.inactive:
        case AppLifecycleState.paused:
        case AppLifecycleState.hidden:
        case AppLifecycleState.detached:
            _ticker?.cancel();
    }
}

/// Foreground smoothing: advance + persist a few times a minute so the bars
/// animate live. The math is identical to the offline path -- it is the same
/// reconcile() call, just over tiny deltas. Tune the interval to taste.
void _startTicker() {
    _ticker?.cancel();
    _ticker = Timer.periodic(const Duration(seconds: 3), (_) => _reconcile());
}

Future<void> _reconcile() async {
    if (!mounted) return;
    await ref.read(robotRepositoryProvider).loadActiveAndReconcile();
}

@override
Widget build(BuildContext context) => widget.child;
}

```

Production note: the 3-second ticker is a verification convenience. In a shipped build you usually do not need a foreground timer at all -- reconcile on resume, then render a projected value on each frame using the pure engine (no DB writes). Easy to swap later; the persistence path is already correct.

## 5. The Pet Screen

### lib/ui/pet\_screen.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../app/providers.dart';
import '../data/models/robot_instance.dart';

class PetScreen extends ConsumerWidget {
    const PetScreen({super.key});

    @override
    Widget build(BuildContext context, WidgetRef ref) {
        final asyncRobot = ref.watch(activeRobotStreamProvider);

        return Scaffold(
            appBar: AppBar(title: const Text('RoboPet')),
            body: asyncRobot.when(
                loading: () => const Center(child: CircularProgressIndicator()),
                error: (e, _) => Center(child: Text('Error: $e')),
                data: (robot) => robot == null
                    ? const Center(child: Text('No active robot'))
                    : _PetView(robot: robot),
            ),
        );
    }
}

```

```

class _PetView extends ConsumerWidget {
  const _PetView({required this.robot});
  final RobotInstance robot;

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final repo = ref.read(robotRepositoryProvider);
    final text = Theme.of(context).textTheme;

    return Padding(
      padding: const EdgeInsets.fromLTRB(20, 16, 20, 24),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          // --- Header: avatar, nickname, level ---
          Row(
            children: [
              CircleAvatar(
                radius: 30,
                backgroundColor: Colors.tealAccent.withOpacity(0.18),
                child: const Icon(Icons.smart_toy, size: 34),
              ),
              const SizedBox(width: 14),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  children: [
                    Text(robot.nickname, style: text.headlineSmall),
                    const SizedBox(height: 2),
                    Text(
                      'Level robot.level - rarity - class',
                      style: text.bodyMedium?.copyWith(color: Colors.white60),
                    ),
                  ],
                ),
              ),
              if (robot.neglectStrikes > 0)
                const Icon(Icons.warning_amber,
                  color: Colors.orangeAccent, size: 18),
            ],
          ),
          const SizedBox(height: 32),

          // --- Care stat bars ---
          _StatBar(
            label: 'Battery',
            icon: Icons.battery_charging_full,
            value: robot.battery.value,
            max: robot.battery.max,
            color: Colors.greenAccent,
          ),
          const SizedBox(height: 18),
          _StatBar(
            label: 'Oil',
            icon: Icons.water_drop,
            value: robot.oil.value,
            max: robot.oil.max,
            color: Colors.amberAccent,
          ),
          const SizedBox(height: 18),
          _StatBar(
            label: 'Entertainment',
            icon: Icons.videogame_asset,
            value: robot.entertainment.value,
            max: robot.entertainment.max,
            color: Colors.pinkAccent,
          ),
          const Spacer(),

```

```

// --- Care actions ---
Row(
  children: [
    Expanded(
      child: _ActionButton(
        label: 'Feed',
        icon: Icons.bolt,
        onTap: () => repo.feed(robot.instanceId),
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: _ActionButton(
        label: 'Oil',
        icon: Icons.water_drop,
        onTap: () => repo.clean(robot.instanceId),
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: _ActionButton(
        label: 'Play',
        icon: Icons.sports_esports,
        onTap: () => repo.play(robot.instanceId),
      ),
    ),
  ],
),
],
),
),
);
}
}

/// Smoothly animates between values whenever the stream emits a new state.
class _StatBar extends StatelessWidget {
  const _StatBar({
    required this.label,
    required this.icon,
    required this.value,
    required this.max,
    required this.color,
  });

  final String label;
  final IconData icon;
  final double value;
  final double max;
  final Color color;

  @override
  Widget build(BuildContext context) {
    final pct = (value / max).clamp(0.0, 1.0);

    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          children: [
            Icon(icon, size: 18, color: color),
            const SizedBox(width: 8),
            Text(label, style: Theme.of(context).textTheme.titleSmall),
            const Spacer(),
            Text('value / max',
              style: Theme.of(context).textTheme.bodySmall),
          ],
        ),
        const SizedBox(height: 6),
      ],
    );
  }
}

```

```

ClipRRect(
  borderRadius: BorderRadius.circular(10),
  // TweenAnimationBuilder animates from the previous value to the new
  // end each rebuild, so bars glide instead of snapping.
  child: TweenAnimationBuilder<double>(
    tween: Tween<double>(begin: 0, end: pct),
    duration: const Duration(milliseconds: 500),
    curve: Curves.easeOut,
    builder: (context, v, _) => LinearProgressIndicator(
      value: v,
      minHeight: 16,
      backgroundColor: color.withOpacity(0.15),
      valueColor: AlwaysStoppedAnimation<Color>(color),
    ),
  ),
),
),
],
);
}
}

class _ActionButton extends StatelessWidget {
  const _ActionButton({
    required this.label,
    required this.icon,
    required this.onTap,
  });

  final String label;
  final IconData icon;
  final Future<void> Function() onTap;

  @override
  Widget build(BuildContext context) {
    return FilledButton.tonalIcon(
      onPressed: () => onTap(),
      icon: Icon(icon),
      label: Text(label),
      style: FilledButton.styleFrom(
        padding: const EdgeInsets.symmetric(vertical: 16),
      ),
    );
  }
}

```

Note: in the Text widgets above, the level/rarity/class line and the "value / max" readout use Dart string interpolation in the live code on the Notion page; they are shown here in plain form to keep the PDF copy clean.

## 6. Verifying decay in real time

The starter rates in RobotFactory are tuned for a real game (battery approx -2.9/hour), so bars barely move live. To watch decay happen on screen, temporarily crank the drain in RobotFactory.starter (e.g. drainPerSecond = 2.0) and run:

```
flutter run
```

Then:



- App launches -> starter pet seeded -> bars render full.
- Bars visibly tick down every 3s (the foreground ticker).
- Tapping Feed / Oil / Play smoothly refills the matching bar.
- Background the app for a minute, reopen -> bars jump down by the offline amount (resume catch-up), exactly matching the live rate.
- Let a bar hit 0 while backgrounded -> neglectStrikes badge appears in the header.

That closes out Phase 1 -- The Living Pet: persistent state, deterministic offline progression, reactive UI, and lifecycle-driven catch-up, all verifiable by hand. Phase 2 (Collection & Gacha) is the next milestone whenever you are ready.